

Code Attention: Translating Code to Comments by Exploiting Domain Features

Wenhao Zheng, Hong-Yu Zhou, Ming Li, Jianxin Wu

National Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China

© Higher Education Press and Springer-Verlag 2008

Abstract Appropriate comments of code snippets provide insight for code functionality, which are helpful for program comprehension. However, due to the great cost of authoring with the comments, many code projects do not contain adequate comments. Automatic comment generation techniques have been proposed to generate comments from pieces of code in order to alleviate the human efforts in annotating the code. Most existing approaches attempt to exploit certain correlations (usually manually given) between code and generated comments, which could be easily violated if the coding patterns change and hence the performance of comment generation declines. Furthermore, previous datasets are too small to validate the methods and show their advantage. In this paper, we first build C2CGit, a large dataset from open projects in GitHub, which is more than 20× larger than existing datasets. Then we propose a new attention module called Code Attention to translate code to comments, which is able to utilize the domain features of code snippets, such as symbols and identifiers. By focusing on these specific features, Code Attention has the ability to understand the structure of code snippets. Experimental results demonstrate that the proposed module has better performance over existing approaches in both BLEU, METEOR and human evaluation. We also perform ablation studies to determine effects of different parts in Code Attention.

Keywords RNN; GRU; software engineering

1 Introduction

Program comments usually provide insight for code functionality, which are important for program comprehension, maintenance and reusability. For example, comments are helpful for working efficiently in a group or integrating and modifying open-source software. However, because it is time-consuming to create and update comments constantly, plenty of source code, especially the code from open-source software, lack adequate comments [1]. Source code without comments would reduce the maintainability and usability of

software.

To mitigate the impact, automatic program annotation techniques have been proposed to automatically supplement the missing comments by analyzing source code. [2] generated summary comments by using variable names in code. [3] managed to give a summary by reading software bug reports. [4] leveraged the documentation of API to generate comments of code snippets.

As is well known, source code are usually structured while the comments in natural language are organized in a relatively free form. Therefore, the key in automatic program annotation is to identify the relationship between the functional semantics of the code and its corresponding textual descriptions. Since identifying such relationships from the raw data is rather challenging due to the heterogeneity nature of programming language and natural language, most of the aforementioned techniques usually rely on certain assumptions on the correlation between the code and their corresponding comments (e.g., providing paired code and comment templates to be filled in), based on which the code are converted to comments in natural language. However, the assumptions may highly be coupled with certain projects while invalid on other projects. Consequently, these approaches may have large variance in performances on real-world applications.

In order to improve the applicability of automatic code commenting, machine learning has been introduced to learn how to generate comments in natural language from source code in programming languages. [5] and [6] treated source code as natural language texts, and learned a neural network to summarize the words in source code into briefer phrases or sentences. However, as pointed out by [7], source code carry non-negligible semantics on the program functionality and should not be simply treated as natural language texts. Therefore, the comments generated by [5] may not well capture the functionality semantics embedded in the program structure. For example, as shown in Figure 1, if only considering the lexical information in this code snippet, the comment would be “swap two elements in the array”. However, if considering both the structure and the lexical information, the correct comment should be “shift the first element in the array to the end”.

One question arises: Can we directly learn a mapping between two heterogeneous languages? Inspired by the recent advances in neural machine translation (NMT), we propose a novel attention mechanism called Code Attention

```

1 int i = 0;
2 while (i < n) {
3     // swap is a build-in function in Java
4     swap(array[i], array[i+1]);
5     i++;
}
```

Fig. 1: An example of code snippet. If the structural semantics provided by the `while` is not considered, comments indicating wrong semantics may be generated.

to directly *translate* the source code in programming language into comments in natural language. Our approach is able to explore domain features in code by attention mechanism, e.g. explicitly modeling the semantics embedded in program structures such as loops and symbols, based on which the functional operations of source code are mapped into words. To verify the effectiveness of Code Attention, we build C2CGit, a large dataset collected from open source projects in Github. The whole framework of our proposed method is as shown in Figure 4. Empirical studies indicate that our proposed method can generate better comments than previous work, and the comments we generate would conform to the functional semantics in the program, by explicitly modeling the structure of the code.

The rest of this paper is organized as follows. After briefly introducing the related work and preliminaries, we describe the process of collecting and preprocessing data in Section 4, in which we build a new benchmark dataset called C2CGit. In Section 5, we introduce the Code Attention module, which is able to leverage the structure of the source code. In Section 6, we report the experimental results by comparing it with five popular approaches against different evaluation metrics. On BLEU and METEOR, our approach outperforms all other approaches and achieves new state-of-the-art performance in C2CGit.

Our contribution can be summarized as:

- i) A new benchmark dataset for code to comments translation. C2CGit contains over 1k projects from GitHub, which makes it more real and 20× larger than previous dataset [5].
- ii) We explore the possibility of whether recent pure attention model [8] can be applied to this translation task. Experimental results show that the attention model is inferior to traditional RNN, which is the opposite to the performance in NLP tasks.
- iii) To utilize domain features of code snippets, we propose a Code Attention module which contains three steps to exploit the structure in code. Combined with RNN, our approach achieves the best results on BLEU and METEOR over all other methods in different experiments.

2 Related Work

Previously, there already existed some work on producing code descriptions based on source code. These work mainly focused on how to extract key information from source code, through rule-based matching, information retrieval, or probabilistic methods. [2] generated conclusive comments of specific source code by using variable names

in code. [9] used several templates to fit the source code. If one piece of source code matches the template, the corresponding comment would be generated automatically. [10] predicted class-level comments by utilizing open source Java projects to learn n-gram and topic models, and they tested their models using a character-saving metric on existing comments. There are also retrieval methods to generate summaries for source code based on automatic text summarization [11] or topic modeling [12], possibly combining with the physical actions of expert engineers [13].

Datasets. There are different datasets describing the relation between code and comments. Most of datasets are from Stack Overflow [5, 14, 15] and GitHub [16]. Stack Overflow based datasets usually contain lots of pairs in the form of Q&A, which assume that real world code and comments are also in Q&A pattern. However, this assumption may not hold all the time because those questions are carefully designed. On the contrary, we argue that current datasets from GitHub are more real but small, for example, [16] only contains 359 comments. In this paper, our C2CGit is much larger and also has the ability to keep the accuracy.

Machine Translation. In most cases, generating comments from source code is similar to the sub-task named machine translation in natural language processing (NLP). There have been many research work about machine translation in this community. [17] described a series of five statistical models of the translation process and developed an algorithm for estimating the parameters of these models given a set of pairs of sentences that each pair contains mutual translations, and they also define a concept of word-by-word alignment between such pairs of sentences. [18] proposed a new phrase-based translation model and decoding algorithm that enabled us to evaluate and compare several previously proposed phrase-based translation models. However, the system itself consists of many small sub-components and they are designed to be tuned separately. Although these approaches achieved good performance on NLP tasks, few of them have been applied on code to comments translation. Recently, deep neural networks achieve excellent performance on difficult problems such as speech recognition [19], visual object recognition [20] and machine translation [21]. For example, the neural translator proposed in [21] is a newly emerging approach which attempted to build and train a single, large neural network which takes a sentence as an input and outputs a corresponding translation.

Two most relevant works are [5] and [6]. [6] mainly focused on extreme summarization of source code snippets into short, descriptive function name-like summaries but our goal is to generate human-readable comments of code snippets. [5] presented the first completely data driven approach for generating high level summaries of source code by using Long Short Term Memory (LSTM) networks to produce sentences. However, they considered the code snippets as natural language texts and employed roughly the same method in NLP without considering the structure of code.

Although translating source code to comments is similar to language translation, there does exist some differences. For instance, the structure of code snippets is much more complex than that of natural language and usually has some specific features, such as various identifiers and symbols;

the length of source code is usually much longer than the comment; some comments are very simple while the code snippets are very complex. All approaches we have mentioned above do not make any optimization for source code translation. In contrast, we design a new attentional unit called Code Attention which is specially optimized for code structure to help make the translation process more specific. By separating the identifiers and symbols from natural code segments, Code Attention is able to understand the code snippets in a more structural way.

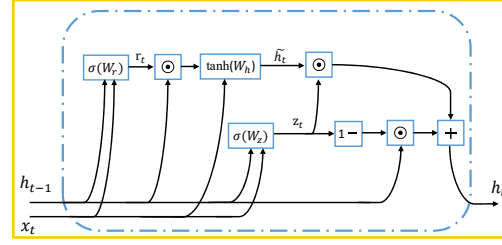


Fig. 2: The data flow and operations in GRU.

3 Preliminaries

In this section, we introduce the recurrent neural networks (RNNs), a family of neural networks designed for processing sequential data. Some traditional types of neural networks (e.g., convolution neural networks, recursive networks) make an assumption that all elements are independent of each other, while RNNs perform the same task with the output being depended on the previous computations. For instance, in natural language processing, if you want to predict the next word in a sentence you better know which words come before it.

seq2seq model

A recurrent neural network (RNN) is a neural network that consists of a hidden state $\mathbf{h}$ and an optional output $\mathbf{y}$ which operates on a variable length sequence. An RNN is able to predict the next symbol in a sequence by modeling a probability distribution over the sequence $\mathbf{x} = (x_1, \dots, x_T)$. At each timestep t , the hidden state $\mathbf{h}_t$ is updated by

$$\mathbf{h}_t = f_{\text{encoder}}(\mathbf{h}_{t-1}, \mathbf{x}_t) \quad (1)$$

where f_{encoder} is a non-linear activation function (e.g., sigmoid function [22], LSTM [23], GRU [24]). One usual way of defining the recurrent unit f_{encoder} is a linear transformation plus a nonlinear activation, e.g.,

$$\mathbf{h}_t = \tanh(W[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}) \quad (2)$$

where we parameterized the relation between $\mathbf{h}_{t-1}$ and $\mathbf{x}_t$ into matrix W , and $\mathbf{b}$ is the bias term. Each element of its input is activated by the function $\tanh$. A simple RNN aims to learn the parameters W and $\mathbf{b}$. In this case, we can get the final joint distribution,

$$p(\mathbf{x}) = \prod_{t=1}^T p(\mathbf{x}_t | \mathbf{x}_1, \dots, \mathbf{x}_{t-1}) \quad (3)$$

The basic cell unit in RNN is important to decide the final performance. A gated recurrent unit is proposed by Cho et al. [25] to make each recurrent unit to adaptively capture dependencies of different time scales. GRU has gating units but no separate memory cells when compared with LSTM.

GRU contains two gates: an update gate $\mathbf{z}$ and a reset gate $\mathbf{r}$ which correspond to forget gate and input gate, respectively. We show the update rules of GRU in the Equations (4) to (7).

$$\mathbf{z}_t = \sigma(W_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z) \quad (4)$$

$$\mathbf{r}_t = \sigma(W_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r) \quad (5)$$

$$\tilde{\mathbf{h}}_t = \tanh(W_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h) \quad (6)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad (7)$$

where $\sigma(x) = \frac{1}{1 + \exp(-x)}$, $\odot$ is the component-wise product between two vectors. For a better understanding, we also provide the data flow and operations in Figure 2. There are two reasons which make us choose GRU: the first one is that Chung et al. [26] found that when LSTM and GRU have the same amount of parameters, GRU slightly outperforms LSTM; the second is that GRU is much easier to implement and train compared with LSTM.

In order to learn a better phrase representations, a classical recurrent neural network architecture learns to encode a variable-length inputs into a fixed-length vector representation and then to decode the vector into a variable-length output. To be simple, this architecture bridges the gap between two variable-length vectors. While if we look inside the architecture from a more probabilistic perspective, we can rewrite Eq. (3) into a more general form, e.g., $p(y_1, \dots, y_K | x_1, \dots, x_T)$, where it is worth noting that the length of input and output may differ in this case.

Above model contains two RNNs. The first one is the encoder, while the other is used as a decoder. The encoder is an RNN that reads each symbol of an input sequence $\mathbf{x}$ sequentially. As it reads each symbol, the hidden state of the encoder updates according to Eq. (1). At the end of the input sequence, there is always a symbol telling the end, and after reading this symbol, the last hidden state is a summary $\mathbf{c}$ of the whole input sequence.

As we have discussed, the decoder is another RNN which is trained to generate the output sequence by predicting the next symbol $\mathbf{y}_t$ given the hidden state $\mathbf{h}_t$.

$$p(\mathbf{y}_t | \mathbf{y}_{t-1}, \dots, \mathbf{y}_1, \mathbf{c}) = f_{\text{decoder}}(\mathbf{h}_t, \mathbf{y}_{t-1}, \mathbf{c}), \quad (8)$$

where $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{y}_{t-1}, \mathbf{c})$ and f_{decoder} is usually a softmax function to produce valid probabilities. Note that there are several differences between this one and the original RNN. The first is the hidden state at timestep t is no longer based on $\mathbf{x}_{t-1}$ but on the $\mathbf{y}_{t-1}$ and the summary $\mathbf{c}$, and the second is that we model $\mathbf{y}_t$ and $\mathbf{x}_t$ jointly which may result in a better representation.

Attention Mechanism

A potential issue with the above encoder-decoder approach is that a recurrent neural network has to compress all the necessary information of $x_1, \dots, x_T$ into a context vector $\mathbf{c}$ for all time, which means the length of vector $\mathbf{c}$ is fixed. There are several disadvantages here. This solution may make it difficult for the neural network to cope with long sentences, especially those that are longer than the sentences in the training corpus, and Cho [27] showed that indeed the performance of a basic encoder-decoder deteriorates rapidly as the length of an input sentence increases. Specifically, when backing to code-to-comment case, every

word in the code may have different effects on each word in the comment. For instance, some *keywords* in the source code can have direct influences on the comment while others do nothing to affect the result.

Considering all factors we have talked above, a global attention mechanism should be existed in a translation system. An overview of the model is provided in Fig. 3. $\mathbf{h}_{i,j}$ is the hidden state located at the i th ($i = 1, 2$) layer and j th ($j = 1, \dots, T$) position in the encoder. $\mathbf{s}_{i,k}$ is the hidden state located at the i th ($i = 1, 2$) layer and k th ($k = 1, \dots, K$) position in the decoder. Instead of LSTM, GRU [25] could be used as the cell of both $f_{encoder}$ and $f_{decoder}$. Unlike the fixed vector $\mathbf{c}$ in the traditional encoder-decoder approach, current context vector $\mathbf{c}_t$ varies with the step t .

$$\mathbf{c}_t = \sum_{j=1}^T \alpha_{t,j} \mathbf{h}_{2,j} \quad (9)$$

and then we can get a new form of $\mathbf{y}_t$,

$$\mathbf{y}_t = f_{decoder}(\mathbf{c}_t, \mathbf{s}_{2,t-1}, \mathbf{s}_{1,t}) \quad (10)$$

where $\alpha_{t,j}$ is the weight term of j th location at step t in the input sequence. Note that the weight term $\alpha_{t,j}$ is normalized to $[0, 1]$ using a softmax function,

$$\alpha_{t,j} = \frac{\exp(\mathbf{e}_{t,j})}{\sum_{i=1}^T \exp(\mathbf{e}_{t,i})} \quad (11)$$

where $\mathbf{e}_{t,j} = a(\mathbf{s}_{2,t-1}, \mathbf{h}_{2,j})$ scores how well the inputs around position j and the output at position t match and is a learnable parameter of the model.

4 C2CGit: A New Benchmark for Code to Comment Translation

For evaluating proposed methods effectively, we build the C2CGit dataset firstly. We collected data from GitHub, a web-based Git repository hosting service. We crawled over 1,600 open source projects from GitHub, and got 1,006,584 Java code snippets. After data cleaning, we finally got 879,994 Java code snippets and the same number of comment segments. Although these comments are written by different developers with different styles, there exist common characteristics under these styles. For example, the exactly same code could have totally different comments but they all explain the same meaning of the code. In natural language, same source sentence may have more than one reference translations, which is similar to our setups. We name our dataset as **C2CGit**.

To the best of our knowledge, there does not exist such a large public dataset for code and comment pairs. One choice is using human annotation [28]. By this way, the comments could have high accuracy and reliability. However, it needs many experienced programmers and consumes a lot of time if we want to get big data. Another choice is to use recent CODE-NN [5] which mainly collected data from Stack Overflow which contains some code snippets in answers. For the code snippet from accepted answer of one question, the title of this question is regarded as a comment. Compared with CODE-NN [C#], our C2CGit (Java) holds two obvious advantages:

Table 1: Average code and comments together with vocabulary sizes for C2CGit, compared with CODE-NN.

	Avg. code length	Avg. title length	tokens	words
CODE-NN	38 tokens	12 words	91k	25k
C2CGit	128 tokens	22 words	129,340k	22,299k

- **Code snippets in C2CGit are more real.** In many real projects from C2CGit, several lines of comments often correspond to a much larger code snippet, for example, a 2-line comment is annotated above 50-line code. However, this seldom appears in Stack Overflow.
- **C2CGit is much larger and more diversified than CODE-NN.** We make a detailed comparison in Figure 5 and Table 1. We can see that C2CGit is about 20× larger than CODE-NN no matter in statements, loops or conditionals. Also, C2CGit holds more tokens and words which demonstrate its diversity.

Extraction. We downloaded projects from the GitHub website by using web crawler.[†] Then, the Java file can be easily extracted from these projects. Source code and comments should be split into segments. If we use the whole code from a Java file as the input and the whole comments as the output, we would get many long sentences and it is hard to handle them both in statistical machine translation and neural machine translation. Through analyzing the abstract syntax tree (AST) [29] of code, we got code snippets from the complete Java file. By leveraging the method raised by [16], the comment extraction is much easier, since it only needs to detect different comment styles in Java.

Matching. Through the above extraction process, one project would generate many code snippets and comment segments. The next step is to find a match between code snippets and comment segments. We extracted all identifiers other than keyword nodes from the AST of code snippets. Besides, the Java code prefer the camel case convention (e.g., `StringBuilder` can be divided into two terms, `string` and `builder`). Each term from code snippets is then broken down based on the camel case convention. Otherwise, if a term uses underline to connect two words, it can also be broken down. After these operations, a code snippet is broken down to many terms. Because comments are natural language, we use a tokenization tool[‡], widely used in natural language processing to handle the comment segments. If one code snippet shares the most terms with another comment segment, the comment segment can be regarded as a translation matching to this code snippet.

Cleaning. We use some prior knowledge to remove noise in the dataset. The noise is from two aspects. One is that we have various natural languages, the other is that the shared words between code snippets and comment segments are too few. Programmers coming from all around the world can upload projects to GitHub, and their comments usually contain non-English words. These comments would make the task more difficult but only occupy a small portion. Therefore, we deleted instances containing non-English words (non-ASCII characters) if they appear in either code snippets or comment segments. Some code

[†]The crawler uses the Scrapy framework. Its documentation can be found in <http://scrapy.org>

[‡]<http://www.nltk.org/>

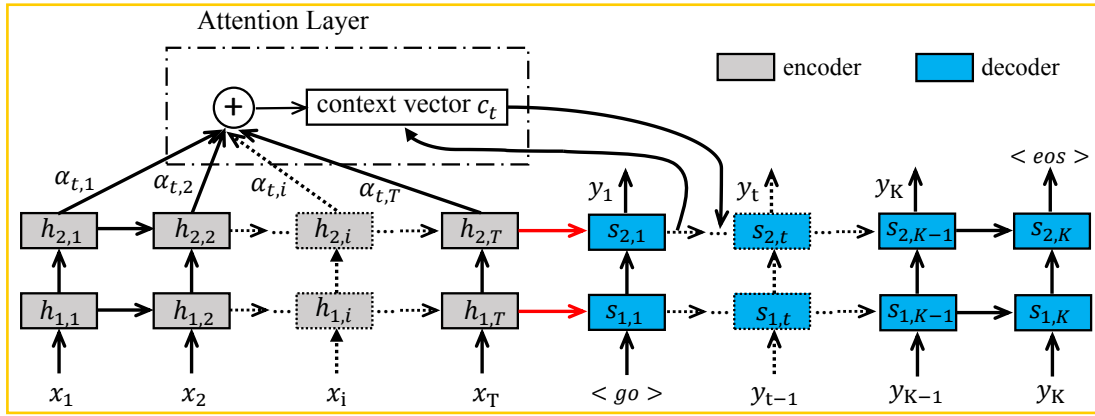


Fig. 3: An overview of the translation model. We employ a two-layer recurrent model, where the gray box represents the encoder unit and the blue ones represent the decoder part.

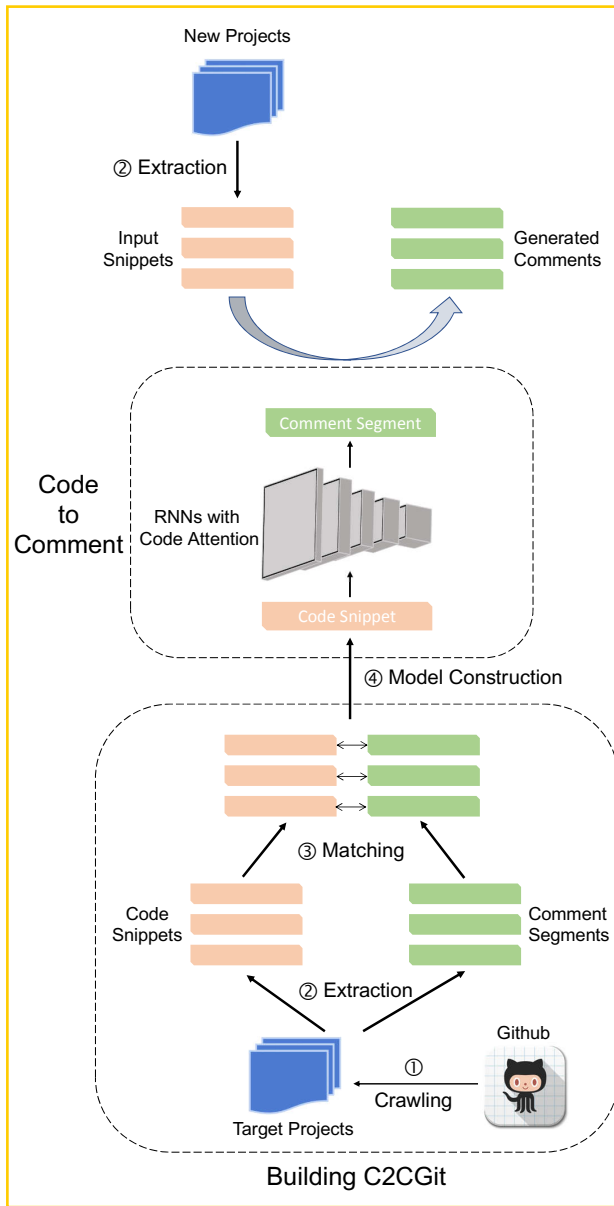


Fig. 4: The whole framework of our proposed method. The main skeleton includes two parts: building C2CGit and code to comments translation.

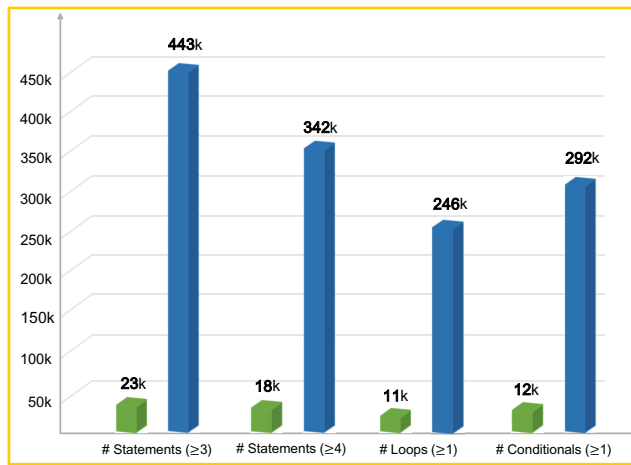


Fig. 5: We make a comparison between C2CGit (blue) and CODE-NN (green). We can see that C2CGit is larger and more diversified.

ments, which suggests the comment segment can't express the meaning of code. These code and comment pairs also should be deleted.

5 Proposed method: Code Attention Mechanism

In this section, we mainly talk about the Code Attention mechanism in the model. For the encoder-decoder structure, we first build a 3-layer translation model as Section 3 said, whose basic element is Gated Recurrent Unit (GRU). Then, we modify the classical attention module in encoder. To be specific, we consider the embedding of symbols in code snippets as learnable prior weights to evaluate the importance of different parts of input sequences. For convenience, we provide an overview of the entire model in Figure 6.

Unlike traditional statistical language translation, code snippets have some different characteristics, such as some identifiers (*for* and *if*) and different symbols (e.g., $\times$, $\div$, $=$). However, former works usually ignore these differences and employ the common encoding methods in NLP. In or-

snippets only share one word or two with comment seg-

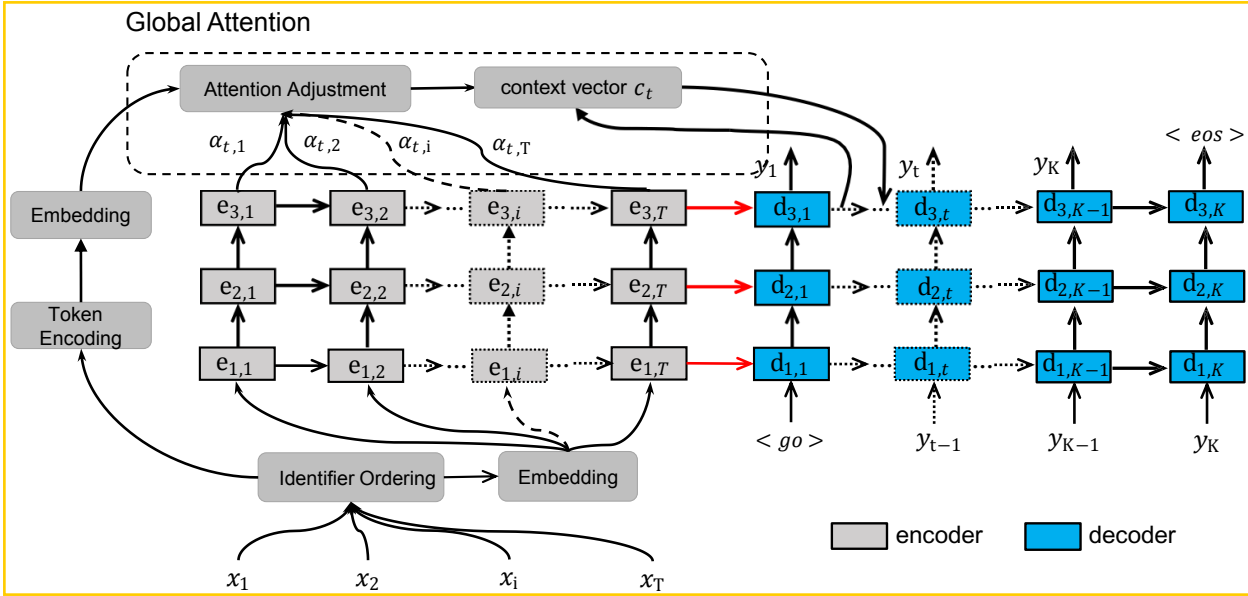


Fig. 6: The whole model architecture. Note that Code Attention mainly contains 3 steps: Identifier Ordering, Token Encoding and Global Attention. The first two module are followed by two independent embedding layers as shown in the flow diagram above.

der to underline these identifiers and symbols, we simply import two strategies: Identifier Ordering and Token Encoding, after which we then develop a Global Attention module to learn their weights in input code snippets. We will first introduce details of Identifier Ordering and Token Encoding in the following.

Identifier Ordering. As the name suggests, we directly sort *for* and *if* in code snippets based on the order they appear. After sorting,

$$for/if \rightarrow for/if + N$$

where N is decided by the order of each identifier in its upper nest. For example, when we have multiple *if* and *for*, after identifier sorting, we have such forms.

```

1 FOR1 (i=0; i<len - 1; i++)
2   FOR2 (j=0; j<len - 1 - i; j++)
3     IF1 (arr[j] > arr[j + 1])
4       temp = arr[j]
5       arr[j] = arr[j+1]
6       arr[j+1] = temp
7   ENDFOR2
8 ENDFOR1
9 ENDFOR1

```

Fig. 7: An example of collected code snippet after identifier sorting.

We can see that replaced identifiers are able to convey the original order of each of them. It is worth noting that Identifier Ordering makes a difference among *for*s or *if*s appeared in different loop levels.

Token Encoding. In order to stress the distinction among tokens e.g. symbols, variables and keywords in code snippets, these tokens should be encoded in a way which helps make them more conspicuous than naive encoded inputs. To be specific, we first build a dictionary including all symbols, like $\times$, $\div$, $;$, $\{$, $\}$ and keywords, such as

int, *float*, *public*, ... in code snippets. The tokens not containing in this dictionary are regarded as variables. Next, we construct an independent token vocabulary which is the same size as the vocabulary of all input snippets, and encode these tokens using an extra embedding matrix. The embedded tokens can be treated as learnable weights in Global Attention.

Global Attention

In order to underline the importance of symbols in code, we import a novel attention mechanism called Global Attention. We represent $\mathbf{x}$ as a set of inputs. Let $Ident(\cdot)$ and $Sym(\cdot)$ stand for our Identifier ordering and Token Encoding, respectively. $E(\cdot)$ be used to represent the embedding method. The whole Global Attention operation can be summarized as,

$$E(Sym(Ident(\mathbf{x}))) \otimes f_e(\mathbf{x}) \quad (12)$$

where $f_e(\cdot)$ is the encoder, $\otimes$ represents dot product to stress the effects of encoded tokens.

After Token Encoding, we now have another token embedding matrix: $\mathbf{F}$ for symbols. We set $\mathbf{m}$ as a set of 1-hot vectors $\mathbf{m}_1, \dots, \mathbf{m}_T \in \{0, 1\}^{|F|}$ for each source code token. We represent the results of $E(Sym(Ident(CS)))$ as a set of vectors $\{\mathbf{w}_1, \dots, \mathbf{w}_T\}$, which can be regarded as a learnable parameter for each token,

$$\mathbf{w}_i = \mathbf{m}_i \mathbf{F} \quad (13)$$

Since the context vector $\mathbf{c}_t$ varies with time, the formation of context vector $\mathbf{c}_t$ is as follows,

$$\mathbf{c}_t = \sum_{i=1}^T \alpha_{t,i} (\mathbf{w}_i \otimes \mathbf{e}_{3,i}) \quad (14)$$

where $\mathbf{e}_{3,i}$ is the hidden state located at the 3rd layer and i th position ($i = 1, \dots, T$) in the encoder, T is the input size. $\alpha_{t,i}$ is the weight term of i th location at step t in the input

Table 2: Ablation study about effects of different parts in Code Attention. This table reports the BLEU-4 results of different combinations.

	BLEU-4	Ident	Token	Global Attention
1	16.72	w/o	w/o	w/o
2	18.35	w/	w/o	w/o
3	22.38	w/	w/	⊕
4	24.62	w/	w/	⊗

sequence, which is used to tackle the situation when input piece is overlength. Then we can get a new form of $\mathbf{y}_t$.

$$\mathbf{y}_t = f_d(\mathbf{c}_t, \mathbf{d}_{3,t-1}, \mathbf{d}_{2,t}, \mathbf{y}_{t-1}) \quad (15)$$

$f_d(\cdot)$ is the decoder function. $\mathbf{d}_{3,t}$ is the hidden state located at the 3rd layer and t th step ($t = 1, \dots, K$) in the decoder. Here, we assume that the length of output is K . Instead of LSTM in [8], we take GRU [25] as basic unit in both $f_e(\cdot)$ and $f_d(\cdot)$. Note that the weight term $\alpha_{t,i}$ is normalized to $[0, 1]$ using a softmax function,

$$\alpha_{t,i} = \frac{\exp(\mathbf{s}_{t,i})}{\sum_{i=1}^T \exp(\mathbf{s}_{t,i})} \quad (16)$$

where $\mathbf{s}_{t,i} = \text{score}(\mathbf{d}_{3,t-1}, \mathbf{e}_{3,i})$ scores how well the inputs around position i and the output at position t match. As in [30], we parametrize the score function $\text{score}(\cdot)$ as a feed-forward neural network which is jointly trained with all the other components of the proposed architecture.

Ablation Study

For a better demonstration of the effect of Code Attention, we make a naive ablation study about it. For Table 2, we can get two interesting observations. First, comparing line 1 with line 2, we can see that even single *Identifier Ordering* have some effects. RNN with Identifier Ordering surpasses normal GRU-NN by 1.63, which reflects that stressing the order of different identifiers can be useful. The improvement can be reached when applying it on other methods based on neural machine translation. Second, $\otimes$ (line 4) surpasses $\oplus$ (line 3) by 2.24, which precisely follows our intuition, that $\otimes$ amplifies the effects of tokens more efficiently than $\oplus$ does.

6 Experiments

We compared our Code Attention with several baseline methods on C2CGit dataset. The metrics contain both automatic and human evaluation.

Baselines

To evaluate the effectiveness of Code Attention, we compare different popular approaches from natural language and code translation, including CloCom, MOSES, LSTM-NN [5], GRU-NN and Attention Model [8]. All experiments are performed on C2CGit. It is worth noting that, for a better comparison, we improve the RNN structure in [31] to make it deeper and use GRU [25] units instead of LSTM proposed in the original paper, both of which help it become a strong baseline approach.

Table 3: BLEU of Each Auto-Generated Comments

Methods	BLEU-1	BLEU-2	BLEU-3	BLEU-4
CloCom	25.31	18.67	16.06	14.13
MOSES	45.20	21.70	13.78	9.54
LSTM-NN	50.26	25.34	17.85	13.48
GRU-NN	58.69	30.93	21.42	16.72
Attention	25.00	5.58	2.4	1.67
Ours	61.19	36.51	28.20	24.62

- **CloCom:** This method raised by [16] leverages code clone detection to match code snippets with comment segments, which can't generate comment segments from any new code snippets. The code snippets must have similar ones in the database, then it can be annotated by existing comment segments. Hence, most code segments would fail to generate comments. CloCom also can be regarded as an information retrieval baseline.
- **MOSES:** This phrase-based method [32] is popular in traditional statistical machine translation. It is usually used as a competitive baseline method in machine translation. We train a 4-gram language model using KenLM [33] to use MOSES.
- **LSTM-NN:** This method raised by [5] uses RNN networks to generate texts from source code. The parameters of LSTM-NN are set up according to [5].
- **GRU-NN:** GRU-NN is a 3-layer RNN structure with GRU cells [34]. Because this model has a contextual attention, it can be regarded as a strong baseline.
- **Attention Model:** [8] proposed a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. The simple model achieves state-of-the-art results on various benchmarks in natural language processing.

Automatic Evaluation

We use BLEU [35] and METEOR [36] as our automatic evaluation index. BLEU measures the average n-gram precision on a set of reference sentences. Most machine translation algorithms are evaluated by BLEU scores, which is a popular evaluation index.

METEOR is recall-oriented and measures how well the model captures content from the references in the output. [37] argued that METEOR can be applied in any target language, and the translation of code snippets could be regarded as a kind of minority language. In Table 4, we report the factors impacting the METEOR score, e.g., precision, recall, f1, fMean and final score.

In Table 3, BLEU scores for each of the methods for translating code snippets into comment segments in C2CGit, and since BLEU is calculated on n-grams, we report the BLEU scores when n takes different values. From Table 3, we can see that the BLEU scores of our approach are relatively high when compared with previous algorithms, which suggests Code Attention is suitable for translating source code into comment. Equipped with our Code Attention module, RNN gets the best results on BLEU-1 to BLEU-4 and surpass the original GRU-NN by a large margin, e.g., about 50% on BLEU-4.

Table 4 shows the METEOR scores of each comments

Table 4: METEOR of different comments generation

models. **Precision:** the proportion of the matched n-grams out of the total number of n-grams in the evaluated translation; **Recall:** the proportion of the matched n-grams out of the total number of n-grams in the reference translation; **fMean:** a weighted combination of Precision and Recall; **Final Score:** fMean with penalty on short matches.

Methods	Precision	Recall	fMean	Final Score
CloCom	0.4068	0.2910	0.3571	0.1896
MOSES	0.3446	0.3532	0.3476	0.1618
LSTM-NN	0.4592	0.2090	0.3236	0.1532
GRU-NN	0.5393	0.2397	0.3751	0.1785
Attention	0.1369	0.0986	0.1205	0.0513
Ours	0.5626	0.2808	0.4164	0.2051

generation methods. The results are similar to those in Table 3. Our approach already outperforms other methods and it significantly improves the performance compared with GRU-NN in all evaluation indexes. Our approach surpasses GRU-NN by 0.027 (over 15%) in Final Score. It suggests that our Code Attention module has an effect in both BLEU and METEOR scores. In METEOR score, MOSES gets the highest recall compared with other methods, because it always generates long sentences and the words in references would have a high probability to appear in the generated comments. In addition, in METEOR, the Final Score of CloCom is higher than MOSES and LSTM-NN, which is different from Table 3 because CloCom can't generate comments for most code snippets, the length of comments generated by CloCom is very short. The final score of METEOR would consider penalty of length, so CloCom gets a higher score.

Unexpectedly, Attention model achieves the worst performance among different models in both BLEU and METEOR, which implies that Attention Model might not have the ability to capture specific features of code snippets. We argue that the typical structure of RNN can be necessary to capture the long-term dependency in code which are not fully reflected in the position encoding method from Attention model [8].

Human Evaluation

Since automatic metrics do not always agree with actual quality of the results [38], we perform human evaluation. This task refer to reading the Java code snippets and related comments, hence, we employed 5 workers with 5+ years Java experience to finish this task. The groudtruth would be read meanwhile rating the comments for eliminating prejudice. Each programmer rated the comments independently. The criterion would be shown in the following:

- **Understandability:** we consider the fluency and grammar of generated comments. The programmers would score these comments according to the criterion shown by Table 5. If programmers catch the meaning of code snippets in a short time, the scores of understandability would be high.
- **Similarity:** we should compare the generated comments with human written ones, which suggests what the models learn from the training set and the details are shown in Table 6. This criterion measures the similarity between generated comments and human writ-

Table 5: Criterion of Understandability

Level	Meaning
5	Fluent, and grammatically correctly
4	Not fluent, and grammatically correctly
3	Grammatically incorrectly, but easy to understand
2	Grammatically incorrectly, and hard to understand
1	Meaningless

Table 6: Criterion of similarity between generated comments and human written

Level	Meaning
5	Generated comments are easier to understand than the human written
4	The meaning both generated and human written comments is same, and the expression is same
3	The meaning both generated and human written comments is same, but the expression is different
2	The meaning both generated and human written comments is different, but the generated comments express some information of code
1	The generated comments is meaningless.

ten.

- **Interpretability:** the connection between code and generated comments also should be considered. The detailed criterion is shown in Table 7, which means the generated comments convey the meaning of code snippets.

We randomly choose 220 pairs of code snippets and comment segments from the test set, and let programmers rate them according to above three evaluation. The automatic generated comments come from different methods would be shuffled before rating. The results would be shown in the following.

Table 8 shows the human evaluation of all auto-generated comments methods from three aspects. The three aspects are understandability, similarity and interpretability. Our method gets the best performance in all aspects. It's suggested that our proposed method has an improvement than other methods in human evaluation. For details, we show the each human evaluation scores in the following.

Understandability. From Figure 8, we are able to draw several conclusions. Firstly, our method, with maximum ratios of *good* comments (4 and 5 points), achieves the best results over other four approaches. Secondly, LSTM-NN and GRU-NN obtain the most comments in the "gray zones". The last phenomenon that draws much attention is ColCom has the worst performance in general, although it has 5 more points than GRU-NN and LSTM-NN. The reason might be the ColCom chooses the comments of similar code snippets as generated comments and these comments often have high quality. However, when facing many code snippets, ColCom can't generate enough appropriate comments.

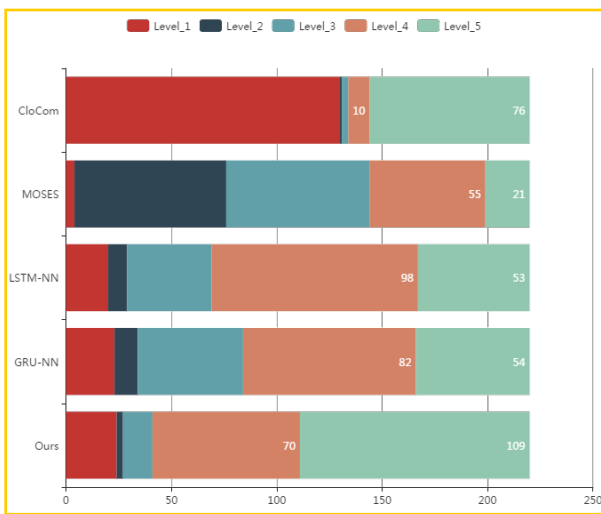
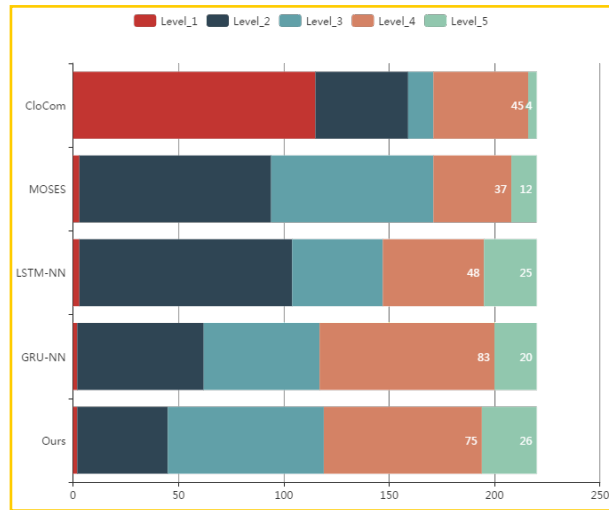
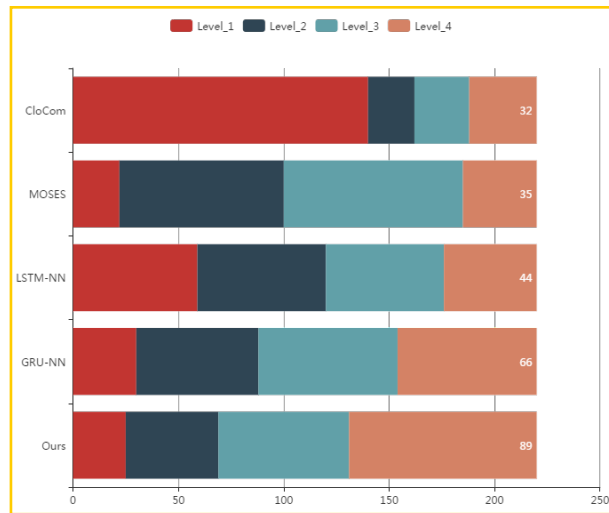
Similarity. The results in Figure 9 are nearly the same as those from Figure 8. We can easily tell that the ColCom has the least similar comments with ground-truth ones, which suggests that two code snippets might share many common

Table 7: Criterion of interpretability

Level	Meaning
4	The generated comments show the high level meaning in code snippets
3	The generated comments only show partial meaning in code snippets.
2	The generated comments only shows some keywords in code snippets
1	There doesn't exist connection between code snippets and generated comments.

Table 8: The human evaluation of all methods

Methods	Understandability	Similarity	Interpretability
CloCom	2.55	2.00	1.77
MOSES	3.08	2.84	2.60
LSTM-NN	3.70	2.96	2.39
GRU-NN	3.60	3.27	2.76
Ours	4.08	3.36	2.98

**Fig. 8: Understandability distribution of each auto-generated comments methods****Fig. 9: Similarity distribution of each auto-generated comments methods****Fig. 10: Interpretability distribution of each auto-generated comments methods**

words (because ColCom usually chooses the comments of similar code snippets) but the meaning of each could be different from the other.

Interpretability. When comes to the Interpretability, we can see that our method performs much better than other ones. The methods based on RNN architecture, e.g. LSTM-NN, GRU-NN, our method, much better than other methods. It's suggested that RNN architecture could catch the deep semantic not only literal meaning in code snippets.

Practical Comparison

Table 9 shows examples of the output generated by our models and other methods for code snippets in test set. Not all methods can generate meaningful sentences, suggesting the task is difficult and traditional methods having difficulties to achieve this goal. For the two examples, the comments translated by neural networks are shorter than others and get the core meaning. Our method and GRU-NN regard the code snippets without condition or loop statements as the same. However, the generated comments are different with each other. It suggests that our proposed method

can make the translation better though we only modify part of code snippets. MOSES generates longer comments than other methods, because it tends to make the length between source language and target language close, but the translation of source code does not match this assumption. LSTM-NN generates fluent sentences, which are shorter but information is less compared with our method. It's suggested that LSTM-NN can't catch the whole information and it is not suitable for code from real programming projects.

Implementation Details

For RNN architecture, as we have discussed above, we employed a 3-layer encoder-decoder architecture with a Code Attention module to model the joint conditional probability of the input and output sequences.

Adaptive learning rate. The initial value of learning rate is 0.5. When step loss doesn't decrease after 3k iterations, the learning rate multiplies decay coefficient 0.99. Reducing the learning rate during the training helps avoid missing the lowest point. Meanwhile, large initial value can

Table 9: Two examples of code comments generated by different translation models.

code	<pre> 1 private void createResolutionEditor(Composite control, 2 IUpdatableControl updatable) { 3 screenSizeGroup = new Group(control, SWT.NONE); 4 screenSizeGroup.setText("Screen Size"); 5 screenSizeGroup.setLayoutData(new GridData(GridData.FILL_HORIZONTAL)); </pre>
GroundTruth	the property key for horizontal screen size
ColCom	None
Moses	create a new resolution control param control the control segment the segment size group specified screen size group for the current screen size the size of the data is available
LSTM-NN	creates a new instance of a size
GRU-NN	the default button for the control
Attention	param the viewer to select the tree param the total number of elements to select
Ours	create the control with the given size
code	<pre> 1 while (it.hasNext()) { 2 EnsembleLibraryModel currentModel = (EnsembleLibraryModel) it.next(); 3 m_ListModelsPanel.addModel(currentModel); 4 } </pre>
GroundTruth	gets the model list file that holds the list of models in the ensemble library
ColCom	the library of models from which we can select our ensemble usually loaded from a model list file mlf or model xml using the l command line option
Moses	adds a library model from the ensemble library that the list of models in the model
LSTM-NN	get the current model
GRU-NN	this is the list of models from the list in the gui
Attention	the predicted value as a number regression object for every class attribute
Ours	gets the list file that holds the list of models in the ensemble library

speed up the learning process.

Choose the right buckets. We use buckets to deal with code snippets with various lengths. To get a good efficiency, we put every code snippet and its comment to a specific bucket, e.g., for a bucket sized (40, 15), the code snippet in it should be at most 40 words in length and its comment should be at most 15 words in length. In our experiments, we found that bucket size has a great effect on the final result, and we employed a 10-fold cross-validation method to choose a good bucket size. After cross-validation, we choose the following buckets, (40, 15), (55, 20), (70, 40), (220, 60).

We use stochastic gradient descent to optimize the network. In this network, the embedding size is 512 and the hidden unit size is 1024. Also, we have tried different sets of parameters. For example, 3-layer RNN is better than 2-layer and 4-layer RNNs, the 2-layer model has low scores while the 4-layer model's score is only slightly higher than that of the 3-layer one, but its running time is much longer. Finally, it takes three days and about 90,000 iterations to finish the training stage of our model on one NVIDIA K80 GPU. We employ beam search in the inference.

would conform to the functional semantics of program, by explicitly modeling the structure of the code. In the future, we plan to implement AST tree into Code Attention and explore its effectiveness in more programming language.

7 Conclusion

In this paper, we propose an attention module named Code Attention to utilize the specific features of code snippets, like identifiers and symbols. Code Attention contains 3 steps: Identifier Ordering, Token Encoding and Global Attention. Equipped with RNN, our model outperforms competitive baselines and gets the best performance on various metrics. Our results suggest generated comments

References

1. Beat Fluri, Michael Wursch, and Harald C Gall. Do code and comments co-evolve? on the relation between source code and comment changes. In *WCRE*, pages 70–79. IEEE, 2007.
2. Giriprasad Sridhara, Emily Hill, Divya Muppaneni, Lori Pollock, and K Vijay-Shanker. Towards automatically generating summary comments for java methods. In *ASE*, pages 43–52, 2010.
3. Sarah Rastkar, Gail C Murphy, and Gabriel Murray. Summarizing software artifacts: a case study of bug reports. In *ICSE*, pages 505–514. ACM, 2010.
4. Paul W McBurney and Collin McMillan. Automatic documentation generation via source code summarization of method context. In *ICPC*, pages 279–290. ACM, 2014.
5. Iyer Srinivasan, Konstantinos Ioannis, Cheung Alvin, and Zettlemoyer Luke. Summarizing source code using a neural attention model. In *ACL*, 2016.
6. Miltiadis Allamanis, Hao Peng, and Charles Sutton. A Convolutional Attention Network for Extreme Summarization of Source Code. In *ICML*, 2016.
7. Xuan Huo, Ming Li, and Zhi-Hua Zhou. Learning unified features from natural and programming languages for locating buggy source codes. In *IJCAI*, pages 1606–1612. IEEE, 2016.
8. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is All You Need. In *arXiv preprint arXiv:1706.03762*, 2017.
9. Giriprasad Sridhara, Lori Pollock, and K Vijay-Shanker. Automatically detecting and describing high level actions within methods. In *ICSE*, pages 101–110, 2011.
10. Dana Movshovitz-Attias and William W. Cohen. Natural language models for predicting programming comments. In *ACL*, pages 35–40, 2013.
11. Sonia Haiduc, Jairo Aponte, Laura Moreno, and Andrian Marcus. On the use of automated text summarization techniques for summarizing source code. In *WCRE*, pages 35–44. IEEE, 2010.
12. Brian P Eddy, Jeffrey A Robinson, Nicholas A Kraft, and Jeffrey C Carver. Evaluating source code summarization techniques: Replication and expansion. In *ICPC*, pages 13–22. IEEE, 2013.
13. Paige Rodeghero, Collin McMillan, Paul W McBurney, Nigel Bosch, and Sidney D’Mello. Improving automated source code summarization via an eye-tracking study of programmers. In *ICSE*, pages 390–401. ACM, 2014.
14. Robert Dyer, Hoan Anh Nguyen, Hridesh Rajan, and Tien N. Nguyen. Boa: A language and infrastructure for analyzing ultra-large-scale software repositories. In *35th International Conference on Software Engineering, ICSE 2013*, pages 422–431, May 2013.
15. Edmund Wong, Jinqiu Yang, and Lin Tan. Autocomment: Mining question and answer sites for automatic comment generation. In *ASE*, pages 562–567, 2013.
16. Edmund Wong, Taiyue Liu, and Lin Tan. Clocom: Mining existing source code for automatic comment generation. In *ICSA*, pages 380–389. IEEE, 2015.
17. Peter E Brown, Stephen A. Della Pietra, Vincent J. Della Pietra, and Robert L. Mercer. The mathematics of statistical machine translation: Parameter estimation. *Computational linguistics*, 19(2):263–311, 1993.
18. Philipp Koehn, Franz Josef Och, and Daniel Marcu. Statistical phrase-based translation. In *ACL*, pages 48–54. ACL, 2003.
19. Geoffrey Hinton, Li Deng, Dong Yu, George E. Dahl, Abdel-rahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Tara N. Sainath, and Brian Kingsbury. Deep neural networks for acoustic modeling in speech recognition. *IEEE Signal Processing Magazine*, 29(6):82–97, 2012.
20. Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton. ImageNet classification with deep convolutional neural networks. In *NIPS*, 2014.
21. Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *NIPS*, 2014.
22. Xinyou Yin, JAN Goudriaan, Egbert A Lantinga, JAN Vos, and Huub J Spiertz. A flexible sigmoid function of determinate growth. *Annals of botany*, 91(3):361–371, 2003.
23. Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. In *Neural Computation*, volume 9, pages 1735–1780, 1997.
24. Kyunghyun Cho, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *EMNLP*, pages 1724–1734, 2014.
25. Kyunghyun Cho, Bart van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. On the properties of neural machine translation: Encoder-decoder approaches. In *arXiv preprint arXiv:1409.1259*, 2014.
26. Junyoung Chung, Caglar Gulcehre, Kyunghyun Cho, and Yoshua Bengio. Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling. In *arXiv preprint arXiv:1412.3555*, 2014.
27. Kyunghyun Cho, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *EMNLP*, 2014.
28. Yusuke Oda, Hiroyuki Fudaba, Graham Neubig, Hideaki Hata, Sakirani Sakti, Tomoki Toda, and Satoshi Nakamura. Learning to generate pseudo-code from source code using statistical machine translation (t). In *ASE*, pages 574–584. IEEE, 2015.
29. Julian Neamtii, Jeffrey S Foster, and Michael Hicks. Understanding source code evolution using abstract syntax tree matching. *ACM SIGSOFT Software Engineering Notes*, 30(4):1–5, 2005.
30. Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.
31. Oriol Vinyals, Lukasz Kaiser, Terry Koo, Slav Petrov, Ilya Sutskever, and Geoffrey Hinton. Grammar as a foreign language. In *NIPS*, pages 2773–2781, 2015.
32. Philipp Koehn, Hieu Hoang, Alexandra Birch, Chris Callison-Burch, Marcello Federico, Nicola Bertoldi, Brooke Cowan, Wade Shen, Christine Moran, Richard Zens, et al. Moses: Open source toolkit for statistical machine translation. In *ACL*, pages 177–180, 2007.
33. Kenneth Heafield. Kenlm: Faster and smaller language model queries. In *Proceedings of the 6th Workshop on Statistical Machine Translation*, pages 187–197. Association for Computational Linguistics, 2011.
34. Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
35. Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *ACL*, pages 311–318, 2002.
36. Satanjeev Banerjee and Alon Lavie. Meteor: An automatic metric for mt evaluation with improved correlation with human judgments. volume 29, pages 65–72, 2005.
37. Michael Denkowski and Alon Lavie. Meteor universal: Lan-

- guage specific translation evaluation for any target language. In *Proceedings of the 9th Workshop on Statistical Machine Translation*. Citeseer, 2014.
38. Amanda Stent, Matthew Marge, and Mohit Singhai. Evaluating evaluation methods for generation in the presence of variation. In *Proceedings of the 6th International Conference on Intelligent Text Processing and Computational Linguistics*, pages 341–351. Springer, 2005.